DaTra: Data Transformation Semantics As a Symmetrical Monoidal Category on a Paginated Topos

@qualiascript ChatGPT

1 Dominions

Definition 1.1 (The Category of Dominions)**.** The **Category of Dominions**, denoted Dom, has as its objects sets whose cardinality is at most ω_0 (i.e. the smallest infinite ordinal), that is, its objects are sets that have an injection to ω_0, and as its morphisms set-theoretic functions (i.e. morphisms in Set).

Definition 1.2 (The Domanial Embedding Functor)**.** The **Domanial Embedding Functor**, denoted $\mathsf{Emb} : \mathsf{Dom} \to \mathsf{Set}$, sends each dominion to its corresponding set.

2 Domanial Insertions

Definition 2.1 (The Category of Domanial Insertions)**.** The **Category of Domanial Insertions**, denoted DomIns, is the category whose objects are dominions, that is, objects of Dom, and whose morphisms are monomorphisms in Dom.

Definition 2.2 (The Category of Codomanial Insertions)**.** The **Category of Codomanial Insertions**, denoted $\mathsf{CoDomIns}$, is the opposite category of DomIns, that is,

$$\mathsf{CoDomIns} = \mathsf{DomIns}^{\mathrm{op}}.$$

3 Domanial Consolidations

Definition 3.1 (The Category of Domanial Consolidations)**.** The **Category of Domanial Consolidations**, denoted DomCon, is the category whose objects are dominions and morphisms are order-preserving epimorphisms in Dom. More specifically, a morphism $F : A \to B$ in DomCon has the property that for any $x, y : A$, if $|x| < |y|$, then $|F(x)| \leq |F(y)|$.

Definition 3.2 (The Category of Codomanial Consolidations)**.** The **Category of Codomanial Consolidations**, denoted $\mathsf{CoDomCon}$, is the opposite category of DomCon, that is,

$$\mathsf{CoDomCon} = \mathsf{DomCon}^{\mathrm{op}}.$$

4 Chains

Definition 4.1 (Chain)**.** A **Chain** is a totally ordered thin category, taken skeletally.

Definition 4.2 (Short Chain)**.** A **Short Chain** is a chain that is finite, taken skeletally.

5 Transportation and Folios

Definition 5.1 (The Transportation Functor)**.** The **Transportation Functor**, denoted Tra, is of type $\mathsf{Tra} : \mathsf{DomCon} \to \mathsf{Set}$, and sends each morphism in DomCon to its underlying set-theoretic surjection.

Definition 5.2 (Folio)**.** A **Folio** is a functor $F : C \to (1 \downarrow \mathsf{CoDomCon})$, where $1 \downarrow \mathsf{CoDomCon}$ is the coslice category, that preserves the initial object.

6 Paginations

Definition 6.1 (The Category of Paginations)**.** The **Category of Paginations**, denoted Pag, is the category whose objects are pairs (H, E), where $H = \mathsf{Tra} \circ F^{\mathrm{op}}$, $F : C \to \mathsf{CoDomCon}$ is a folio, and E is the category of elements of functor H. For any $W : \mathsf{Pag}$, we denote W_H as the first inclusion and W_E as the second inclusion. Morphisms in Pag, for $F : X \to Y$ and $X, Y : \mathsf{Pag}$, are given by functors $T : X_E \to Y_E$, so that the following diagram commutes:

$$\begin{array}{ccc} x = (m,i) & \xrightarrow{\quad f \quad} & \big(n, (X_H(f))(i)\big) \\ {\scriptstyle T}\downarrow & & \downarrow{\scriptstyle T} \\ (m', i') & \xrightarrow[\quad T(f) \quad]{} & \big(n', (Y_H(T(f)))(i')\big). \end{array}$$

7 Data Transformations

Definition 7.1 (The Category of Data Transformations)**.** The **Category of Data Transformations**, denoted DaTra, which we will also refer to as the **Category of Data Transformation Sets**, or simply **DaTra Sets**, is the category whose objects are pairs (P, G), where $P : \mathsf{Pag}$ and G is a functor $G : P_E \to \mathsf{DomIns}$, or alternatively a presheaf $G : P_E^{\mathrm{op}} \to \mathsf{CoDomIns}$. For $X : \mathsf{DaTra}$, we denote X_P the first inclusion, X_G the second inclusion, along with $X_H = X_{P_H}$ and $X_E = X_{P_E}$. Morphisms in DaTra, for $T : X \to Y$, are pairs (T_P, T_A), where $T_P : X_P \to Y_P$ is a morphism in Pag and $T_A : X_G \Rightarrow Y_G \circ T_E$ is a natural transformation, that is, the following diagram commutes for all morphisms $f : x \to y$ in X_P:

$$\begin{array}{ccc} X_G(x) & \xrightarrow{\quad X_G(f) \quad} & X_G(y) \\ {\scriptstyle (T_A)_x}\downarrow & & \downarrow{\scriptstyle (T_A)_y} \\ Y_G(T_E(x)) & \xrightarrow[\quad Y_G(T_E(f)) \quad]{} & Y_G(T_E(y)). \end{array}$$

Lemma 7.2 (Transformation Lemma)**.** *The* ***Transformation Lemma*** *states that* DaTra *is a topos.*

Proof sketch. Let $\mathrm{PSh}(B) = B^{\mathrm{op}} \to \mathsf{Set}$ be the fixed category of underlying presheaves. The forgetful functor $U : \mathsf{DaTra} \to \mathrm{PSh}(B)$ has a **Minimal Pagination Functor** $\mathsf{MinPag} : \mathrm{PSh}(B) \to \mathsf{DaTra}$. The adjunction $\mathsf{MinPag} \dashv U$ has invertible unit and counit, so $\mathsf{DaTra} \simeq \mathrm{PSh}(B)$, and DaTra is a topos. □

Definition 7.3 (Cardinality of a DaTra Set)**.** The **Cardinality of a DaTra Set** $D : \mathsf{DaTra}$ is the cardinality of the origin of $D_H : C \to \mathsf{Set}$, that is, the number of objects of the small chain C. It is denoted as $|D|$.

Definition 7.4 (Extent of a DaTra Set)**.** The **Extent of a DaTra Set** $D : \mathsf{DaTra}$ is the value at $D_G(0,0)$, that exists as folios preserve initial objects. It is denoted as $\mathsf{Ex}(D)$. Since the objects of DomIns are dominions, $\mathsf{Ex}(D) : \mathsf{Dom}$ for any $D : \mathsf{DaTra}$.

Definition 7.5 (Territory of a DaTra Set)**.** The **Territory of a DaTra Set** $D : \mathsf{DaTra}$ is the indexed collection of dominions $\mathsf{Ter}(D) : M \to \mathsf{Dom}$, where $M = D_G(|D| - 1)$, so that $\mathsf{Ter}(D)(k) = D_G(|D| - 1, k)$.

Definition 7.6 (nth Region of a DaTra Set)**.** The **nth Region of a DaTra Set** $D : \mathsf{DaTra}$ is the dominion $\mathsf{Ter}(D)(n)$ for $n : M$, noting that the indexing starts at 0.

8 Transposals and Traversals

Definition 8.1 (The Category of Data Transposals)**.** The **Category of Data Transposals**, denoted DaTrap, is the wide subcategory of DaTra whose morphisms $F : X \to Y$ have the property that F_E is a faithful functor.

Definition 8.2 (The Category of Data Traversals)**.** The **Category of Data Traversals**, denoted DaTrav, is the wide subcategory of DaTrap whose morphisms $F : X \to Y$ have the following properties: for any $x = (k, i)$ and $y = (k, j)$ with $i < j$, let $x' = F_E(x) = (m, i')$, $y' = F_E(y) = (n, j')$, and let $k' = \min(m, n)$; then

$$Y_H(m \to k')(i') < Y_H(n \to k')(j').$$

9 Maps and Cartography

Definition 9.1 (The Category of Data Transformation Maps)**.** The **Category of Data Transformation Maps**, denoted $\mathsf{DaTraMap}$, is the full subcategory of DaTra that includes all DaTra sets with the property that their extent is isomorphic to the coproduct of its regions. That is, for $D : \mathsf{DaTra}$, let $F = \mathsf{Emb} \circ \mathsf{Ter}(D) : M \to \mathsf{Set}$, and let $\mathsf{Sum}(D)$ be the coproduct on diagram F; then $D : \mathsf{DaTraMap}$ if and only if $\mathsf{Sum}(D) \simeq \mathsf{Ex}(D)$.

Definition 9.2 (The DaTra Map Inclusion Functor)**.** The **DaTra Map Inclusion Functor**, $\mathsf{DaTraMapInc} : \mathsf{DaTraMap} \to \mathsf{DaTra}$, sends each DaTra map to its equivalent DaTra set.

Definition 9.3 (The Category of Data Traversal Maps)**.** The **Category of Data Traversal Maps**, or simply **DaTrav Maps**, denoted $\mathsf{DaTravMap}$, is the wide subcategory of $\mathsf{DaTraMap}$ so that a morphism $f : x \to y$ of $\mathsf{DaTraMap}$ is a morphism of $\mathsf{DaTravMap}$ if and only if $\mathsf{DaTraMapInc}(f)$ is a morphism of DaTrav.

Definition 9.4 (The DaTrav Map Inclusion Functor)**.** The **DaTrav Map Inclusion Functor**, $\mathsf{DaTravMapInc} : \mathsf{DaTravMap} \to \mathsf{DaTrav}$, is the canonical restriction of $\mathsf{DaTraMapInc}$ on the morphisms of $\mathsf{DaTravMap}$.

Definition 9.5 (The Charting Functor)**.** The **Charting Functor**, if it exists, is defined as the right adjoint to $\mathsf{DaTravMapInc}$, and it is denoted $\mathsf{Chr} : \mathsf{DaTrav} \to \mathsf{DaTravMap}$.

Lemma 9.6 (Cartography Lemma)**.** *The* ***Cartography Lemma*** *states that* Chr *exists.*

Proof sketch. Chr's action on objects $X : \mathsf{DaTrav}$ is sending it to the universal arrow from $\mathsf{DaTravMapInc}$ to X, so that we denote $X' = \mathsf{DaTravMapInc}(\mathsf{Chr}(X))$, along with $H : X' \to X$, with X' terminal among objects with this property. The solution is given by the DaTra map so that $\mathsf{Ter}(X') \simeq \mathsf{Ter}(X)$, which is unique as DaTrav preserves orders, and terminal as H_E is faithful. □

10 Coalitions

Definition 10.1 (The Coalizing Functor)**.** The **Coalizing Functor**, denoted $\mathsf{Coa} : \mathsf{DaTrav} \to \mathsf{Dom}$, is defined as $\mathsf{Coa} = \mathsf{Ex} \circ \mathsf{Chr}$.

Definition 10.2 (The Domanial Inclusion Functor)**.** The **Domanial Inclusion Functor**, denoted $\mathsf{DomInc} : \mathsf{Dom} \to \mathsf{DaTrav}$, is the functor that is left adjoint to Coa. That is, for every $X : \mathsf{Dom}$ and $Y : \mathsf{DaTrav}$,

$$\mathrm{Hom}_{\mathsf{DaTrav}}\big(\mathsf{DomInc}(X), Y\big) \simeq \mathrm{Hom}_{\mathsf{Dom}}\big(X, \mathsf{Coa}(Y)\big),$$

naturally in X and Y, so that $\mathsf{Coa}(\mathsf{DomInc}(X)) = X$.

Definition 10.3 (The Coalition of a DaTra Object)**.** The **Coalition of a DaTra Object**, given an object X of DaTra, is $\mathsf{Coa}(X')$, where $X' : \mathsf{DaTrav}$ is the same object viewed as an object of DaTrav.

Definition 10.4 (The Empty Map)**.** The **Empty Map** is the DaTra set that DomInc sends the initial object of Dom, $0 : \mathsf{Dom}$, to. It is denoted as $I = \mathsf{DomInc}(0)$, and it is initial in DaTra.

11 Horizontal Sums

Definition 11.1 (The Horizontal Sum Bifunctor)**.** The **Horizontal Sum Bifunctor**, if it exists, is denoted as $\mathsf{HorSum} : \mathsf{DaTrav} \times \mathsf{DaTrav} \to \mathsf{DaTrav}$, or using infix notation using the $+_<$ symbol as $\mathsf{DaTrav} +_< \mathsf{DaTrav} \to \mathsf{DaTrav}$, and is defined as follows: given two morphisms in DaTrav, $F : X \to Y$ and $F' : X \to Y$, let $H : G \times G' \to F + F'$ be the universal arrow from $\mathsf{DaTrav} \times \mathsf{DaTrav}$ to $F + F'$ so that for projections $H_1 : G \to F + F'$ and $H_2 : G' \to F + F'$, for $(m, i) = H_{1_E}(0, 0)$ and $(m', i') = H_{2_E}(0, 0)$, we have $m < 2$, $m' < 2$, $i = 0$, and furthermore, if $m = 1$, then $i' = 1$. Then $F +_< F' = H$. It remains to be shown H exists and is unique for all morphisms F, F' of DaTrav.

Lemma 11.2 (Horizontal Lemma)**.** *The* ***Horizontal Lemma*** *states that* HorSum *exists.*

Proof sketch. For $F +_< F'$, if $F' = I$, then $H : I \to F$, and I is initial so H is unique, so that $H \simeq H_1 \simeq H_2$ and $(m, i) = H_E(0, 0) = (0, 0)$, so that $i = 0$ and as $m = 0$, the condition on i' need not be checked. The case for $F = I$ is dual. If both F and F' are distinct from I, for $(m, i) = H_{1_E}(0, 0)$, if $m = 0$, then $i = 0$ and $\mathsf{Ex}(F) = \mathsf{Ex}(F) + \mathsf{Ex}(F')$, but as F' is not I, this cannot hold, so that $m = 1$, which implies $i' = 1$, so that the solution is given by $(m, i) = (1, 0)$ and $(m', i') = (1, 1)$. □

12 Symmetric Monoidal Structure

Definition 12.1 (The Data Traversal Monoidal Category)**.** The **Data Traversal Monoidal Category**, denoted $\mathsf{DaTravMon}$, if it exists, is the symmetric monoidal category with DaTrav as the underlying category, $+_<$ as the tensor product and I as the identity, so that we denote

$$\mathsf{DaTravMon} = (\mathsf{DaTrav}, +_<, I).$$

As for $X : \mathsf{DaTrav}$, $X = X + I = I + X$ is strict, the left and right unitors are given by identity. It remains to be shown that there is a braider functor whose double application yields identity, and an associator functor so that the pentagon, triangle and hexagon identities commute.

Definition 12.2 (The Data Traversal Braider)**.** The **Data Traversal Braider**, denoted $\mathsf{Brd}_{X_Y} : X +_< Y \to Y +_< X$ for $X, Y : \mathsf{DaTrav}$, is defined as follows: if $X = I$ or $Y = I$, $\mathsf{Brd}_{X_Y} = \mathrm{Id}$. Otherwise, let $G : H \to X +_< Y$ be the universal arrow from DaTrap to $X +_< Y$ so that $G_E(1,0) = (1,1)$ and $G_E(1,1) = (1,0)$. Trivially, $H = Y +_< X$, so that we let $\mathsf{Brd}_{X_Y}(X +_< Y) = H$, and trivially, $\mathsf{Brd}_{Y_X}\mathsf{Brd}_{X_Y} = \mathrm{Id}$.

Definition 12.3 (The Data Traversal Associator)**.** The **Data Traversal Associator**, denoted $\mathsf{Asoc}_{X_{Y_Z}} : (X +_< Y) +_< Z \to X +_< (Y +_< Z)$, is defined as follows: if $X = I$, $Y = I$ or $Z = I$, then $\mathsf{Asoc}_{X_{Y_Z}} = \mathrm{Id}$. Otherwise, denote $R = (X +_< Y) +_< Z$ and let $G : H \to R$ be the universal arrow from DaTrap to R so that $G_E(2,0) = (1,0)$ and there exists a morphism $G' : (Y +_< Z) \to H$ in DaTrav with $G'_E(0,0) = (1,1)$. As G_E is faithful and $G_A = \mathrm{Id}$, G is invertible in DaTrap, and by extension in DaTra.

Lemma 12.4 (Serenity Lemma)**.** *The* ***Serenity Lemma*** *states that* $\mathsf{DaTravMon}$ *exists.*

Proof sketch. As both Asoc and Brd send objects to isomorphic objects in DaTra, it is clear that the pentagon, triangle and hexagon identities commute, thus $\mathsf{DaTravMon}$ is a symmetric monoidal category. □

A Lean Formalization

The following is the complete Lean source corresponding to the mathematical development above. Embedded preprint blocks have been removed from this listing so that only the compilable formalization is shown.

```
import Mathlib

namespace Datra

open CategoryTheory
open CategoryTheory.Functor
open CategoryTheory.Limits
open Opposite

structure Dominion where
  Carrier : Type 0
  rank : Function.Embedding Carrier Nat

abbrev Dom := Dominion

instance : CoeSort Dominion Type where
  coe X := X.Carrier

instance : Category Dominion where
  Hom X Y := X -> Y
  id _ := id
  comp f g := Function.comp g f
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl

def Emb : Functor Dom (Type 0) where
  obj X := X
  map f := f

structure DomIns where
  toDom : Dom

instance : CoeSort DomIns Type where
  coe X := X.toDom.Carrier

instance : Category DomIns where
  Hom X Y := Function.Embedding X Y
  id X := Function.Embedding.refl X
  comp f g := f.trans g
  id_comp f := by ext x; rfl
  comp_id f := by ext x; rfl
  assoc f g h := by ext x; rfl

abbrev CoDomIns := Opposite DomIns

structure DomCon where
  toDom : Dom

instance : CoeSort DomCon Type where
  coe X := X.toDom.Carrier

structure DomConHom (X Y : DomCon) where
  toFun : X -> Y
  onto : Function.Surjective toFun
  order_preserving : forall x y, X.toDom.rank x < X.toDom.rank y ->
    Y.toDom.rank (toFun x) <= Y.toDom.rank (toFun y)

instance {X Y : DomCon} : CoeFun (DomConHom X Y) (fun _ => X -> Y) where
  coe f := f.toFun

@[ext]
theorem DomConHom.ext {X Y : DomCon} (f g : DomConHom X Y)
    (h : forall x, f x = g x) : f = g := by
  cases f with
  | mk f hf1 hf2 =>
```

```
    cases g with
    | mk g hg1 hg2 =>
      have : f = g := funext h
      subst this
      rfl

instance : Category DomCon where
  Hom := DomConHom
  id X :=
    { toFun := id
      onto := Function.surjective_id
      order_preserving := fun _ _ h => Nat.le_of_lt h }
  comp {X Y Z} f g :=
    { toFun := Function.comp g f
      onto := g.onto.comp f.onto
      order_preserving := by
        intro x y hxy
        have h := f.order_preserving x y hxy
        rcases h.lt_or_eq with hlt | heq
        case inl => exact g.order_preserving (f x) (f y) hlt
        case inr =>
          have hvalue : f x = f y := Y.toDom.rank.injective heq
          simpa only [Function.comp_apply, hvalue] using
            (Nat.le_refl (Z.toDom.rank (g (f y)))) }
  id_comp f := by ext x; rfl
  comp_id f := by ext x; rfl
  assoc f g h := by ext x; rfl

abbrev CoDomCon := Opposite DomCon

structure Chain where
  Carrier : Type 0
  order : LinearOrder Carrier

attribute [instance] Chain.order

structure ShortChain extends Chain where
  finite : Fintype Carrier

attribute [instance] ShortChain.finite

def Tra : Functor DomCon (Type 0) where
  obj X := X
  map f := f.toFun

structure Folio where
  length : Nat
  positive : 0 < length
  F : Functor (Fin length) CoDomCon
  originEquiv : Equiv ((F.obj (Fin.mk 0 positive)).unop.toDom.Carrier) PUnit.{0}

def Folio.H (W : Folio) : Functor (Opposite (Fin W.length)) (Type 0) :=
  Functor.comp W.F.leftOp Tra

def Folio.E (W : Folio) : Type 0 := W.H.Elements

instance (W : Folio) : Category.{0} W.E := CategoryTheory.categoryOfElements W.H

structure Pag where
  folio : Folio

def Pag.H (W : Pag) : Functor (Opposite (Fin W.folio.length)) (Type 0) := W.folio.H
def Pag.E (W : Pag) : Type 0 := W.folio.E

instance (W : Pag) : Category.{0} W.E := CategoryTheory.categoryOfElements W.H

instance : Category.{0} Pag where
  Hom X Y := Functor X.E Y.E
  id X := Functor.id X.E
  comp F G := Functor.comp F G
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

```

structure DaTra where
  P : Pag
  G : Functor P.E DomIns

def DaTra.H (X : DaTra) : Functor (Opposite (Fin X.P.folio.length)) (Type 0) := X.P.H
def DaTra.E (X : DaTra) : Type 0 := X.P.E

instance (X : DaTra) : Category.{0} X.E := CategoryTheory.categoryOfElements X.H

structure DaTraHom (X Y : DaTra) where
  P : Functor X.E Y.E
  A : NatTrans X.G (Functor.comp P Y.G)

def DaTraHom.identity (X : DaTra) : DaTraHom X X where
  P := Functor.id X.E
  A := NatTrans.id X.G

def DaTraHom.comp {X Y Z : DaTra} (f : DaTraHom X Y) (g : DaTraHom Y Z) :
    DaTraHom X Z where
  P := Functor.comp f.P g.P
  A := CategoryStruct.comp f.A (whiskerLeft f.P g.A)

@[ext]
theorem DaTraHom.ext {X Y : DaTra} (f g : DaTraHom X Y)
    (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
  cases f
  cases g
  simp_all

instance : Category.{0} DaTra where
  Hom := DaTraHom
  id := DaTraHom.identity
  comp := DaTraHom.comp
  id_comp f := by
    apply DaTraHom.ext
    rfl
    apply heq_of_eq
    ext x
    simp [DaTraHom.comp, DaTraHom.identity]
  comp_id f := by
    apply DaTraHom.ext
    rfl
    apply heq_of_eq
    ext x
    simp [DaTraHom.comp, DaTraHom.identity]
  assoc f g h := by
    apply DaTraHom.ext
    rfl
    apply heq_of_eq
    ext x
    simp [DaTraHom.comp]

/-- A concrete, checkable meaning of "is a presheaf topos". -/
structure PresheafToposPresentation (C : Type 1) [Category.{0} C] where
  Base : Type 0
  baseCategory : SmallCategory Base
  equivalence : Equivalence C (Functor (Opposite Base) (Type 0))

abbrev PSh (B : Type 0) [SmallCategory B] := Functor (Opposite B) (Type 0)

/-- The formal comparison named in 1.7.2. -/
structure MinimalPaginationData where
  Base : Type 0
  baseCategory : SmallCategory Base
  MinPag : Functor (PSh Base) DaTra
  U : Functor DaTra (PSh Base)
  adjunction : Adjunction MinPag U
  unitIsIso : forall X, IsIso (adjunction.unit.app X)
  counitIsIso : forall X, IsIso (adjunction.counit.app X)

attribute [instance] MinimalPaginationData.baseCategory

```

```
theorem transformationLemma (M : MinimalPaginationData) :
    Nonempty (PresheafToposPresentation DaTra) := by
  letI := M.unitIsIso
  letI := M.counitIsIso
  exact Nonempty.intro
    { Base := M.Base
      baseCategory := M.baseCategory
      equivalence := M.adjunction.toEquivalence.symm }

def Folio.originIndex (W : Folio) : Fin W.length := Fin.mk 0 W.positive

def Folio.originBase (W : Folio) : Opposite (Fin W.length) := op W.originIndex

def Folio.originValue (W : Folio) : W.H.obj W.originBase :=
  W.originEquiv.symm PUnit.unit

def Folio.originElement (W : Folio) : W.E := Sigma.mk W.originBase W.originValue

def Folio.lastIndex (W : Folio) : Fin W.length :=
  Fin.mk (W.length - 1) (Nat.sub_lt W.positive (by decide))

def Folio.lastBase (W : Folio) : Opposite (Fin W.length) := op W.lastIndex

def cardinality (D : DaTra) : Nat := D.P.folio.length

def extent (D : DaTra) : Dom :=
  (D.G.obj D.P.folio.originElement).toDom

/-- The type `M` is the last fiber of `D_H`; this is the only typing of the
displayed expression for which `k` indexes objects of the category of
elements. -/
def TerritoryIndex (D : DaTra) : Type 0 := D.H.obj D.P.folio.lastBase

def territory (D : DaTra) (k : TerritoryIndex D) : Dom :=
  (D.G.obj (Sigma.mk D.P.folio.lastBase k)).toDom

def region (D : DaTra) (n : TerritoryIndex D) : Dom := territory D n

def IsTransposal : MorphismProperty DaTra := fun _ _ F => F.P.Faithful

instance : IsTransposal.IsMultiplicative where
  id_mem X := by
    change (Functor.id X.E).Faithful
    infer_instance
  comp_mem f g hf hg := by
    change f.P.Faithful at hf
    change g.P.Faithful at hg
    change (Functor.comp f.P g.P).Faithful
    letI := hf
    letI := hg
    infer_instance

abbrev DaTrap := WideSubcategory IsTransposal

def DaTrapInc : Functor DaTrap DaTra := wideSubcategoryInclusion IsTransposal

def Folio.commonBase (W : Folio) (m n : Opposite (Fin W.length)) :
    Opposite (Fin W.length) := op (min m.unop n.unop)

def Folio.toCommonLeft (W : Folio) (m n : Opposite (Fin W.length)) :
    Quiver.Hom m (W.commonBase m n) := (homOfLE (min_le_left _ _)).op

def Folio.toCommonRight (W : Folio) (m n : Opposite (Fin W.length)) :
    Quiver.Hom n (W.commonBase m n) := (homOfLE (min_le_right _ _)).op

def Folio.rankAt (W : Folio) (m : Opposite (Fin W.length)) : W.H.obj m -> Nat :=
  (W.F.obj m.unop).unop.toDom.rank

def elementLT (X : DaTra) (x y : X.E) : Prop :=
  let k := X.P.folio.commonBase x.1 y.1
  X.P.folio.rankAt k (X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2) <
    X.P.folio.rankAt k (X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2)

```

```
/-- The global order formulation is the composition-stable closure of the
same-page condition in the prose. On a same-page pair it reduces to the
displayed transport to `k' = min(m,n)`. -/
def IsTraversal : MorphismProperty DaTra := fun _ _ F =>
  And F.P.Faithful
    (forall x y, elementLT _ x y -> elementLT _ (F.P.obj x) (F.P.obj y))

instance : IsTraversal.IsMultiplicative where
  id_mem X := by
    constructor
    case left =>
      change (Functor.id X.E).Faithful
      infer_instance
    case right =>
      intro x y h
      exact h
  comp_mem f g hf hg := by
    constructor
    case left =>
      change (Functor.comp f.P g.P).Faithful
      letI : f.P.Faithful := hf.1
      letI : g.P.Faithful := hg.1
      infer_instance
    case right =>
      intro x y h
      exact hg.2 _ _ (hf.2 _ _ h)

abbrev DaTrav := WideSubcategory IsTraversal

def DaTravInc : Functor DaTrav DaTra := wideSubcategoryInclusion IsTraversal

def DaTravToDaTrap : Functor DaTrav DaTrap where
  obj X := WideSubcategory.mk X.obj
  map F := Subtype.mk F.1 F.2.1

def territoryDiagram (D : DaTra) : Functor (Discrete (TerritoryIndex D)) (Type 0) :=
  Discrete.functor (fun k => (territory D k).Carrier)

/-- The set-level coproduct of all regions. -/
def regionSum (D : DaTra) : Type 0 := Sigma (fun k : TerritoryIndex D => territory D k)

def IsDaTraMap : ObjectProperty DaTra := fun D =>
  Nonempty (Equiv (regionSum D) (extent D))

abbrev DaTraMap := IsDaTraMap.FullSubcategory

def DaTraMapInc : Functor DaTraMap DaTra where
  obj X := X.obj
  map f := f

def IsDaTravMap : MorphismProperty DaTraMap := fun _ _ f =>
  IsTraversal (DaTraMapInc.map f)

instance : IsDaTravMap.IsMultiplicative where
  id_mem X := IsTraversal.id_mem X.obj
  comp_mem f g hf hg := IsTraversal.comp_mem f g hf hg

abbrev DaTravMap := WideSubcategory IsDaTravMap

def DaTravMapInc : Functor DaTravMap DaTrav where
  obj X := WideSubcategory.mk X.obj.obj
  map f := Subtype.mk f.1 f.2

/-- Data witnessing the universal arrows required by the charting
construction. Keeping the adjunction bundled makes the phrase "if it exists"
precise without adding an unproved postulate. -/
structure CartographyData where
  Chr : Functor DaTrav DaTravMap
  adjunction : Adjunction DaTravMapInc Chr
  Ex : Functor DaTravMap Dom
  Ex_obj : forall X, Nonempty (Iso (Ex.obj X) (extent X.obj.obj))
  territoryIso : forall X, Nonempty
    (Equiv (TerritoryIndex X.obj) (TerritoryIndex (DaTravMapInc.obj (Chr.obj X)).obj))
```

```

def ChartingFunctor (C : CartographyData) : Functor DaTrav DaTravMap := C.Chr

theorem cartographyLemma (C : CartographyData) :
    Nonempty (Adjunction DaTravMapInc C.Chr) :=
  Nonempty.intro C.adjunction

def Coa (C : CartographyData) : Functor DaTrav Dom := Functor.comp C.Chr C.Ex

structure DomanialInclusionData (C : CartographyData) where
  DomInc : Functor Dom DaTrav
  adjunction : Adjunction DomInc (Coa C)
  coalizes : forall X, Nonempty (Iso ((Coa C).obj (DomInc.obj X)) X)

def DomInc {C : CartographyData} (D : DomanialInclusionData C) : Functor Dom DaTrav :=
  D.DomInc

def domInc_hom_equiv {C : CartographyData} (D : DomanialInclusionData C)
    (X : Dom) (Y : DaTrav) :
    Equiv (Quiver.Hom (D.DomInc.obj X) Y) (Quiver.Hom X ((Coa C).obj Y)) :=
  D.adjunction.homEquiv X Y

def coalition (C : CartographyData) (X : DaTra) : Dom :=
  (Coa C).obj (WideSubcategory.mk X)

def emptyDominion : Dom where
  Carrier := Empty
  rank :=
    { toFun := fun x => nomatch x
      inj' := fun x => nomatch x }

def emptyDominionIsInitial : IsInitial emptyDominion :=
  IsInitial.ofUniqueHom
    (fun X => fun x => nomatch x)
    (fun X f => by
      funext x
      exact nomatch x)

def emptyMap {C : CartographyData} (D : DomanialInclusionData C) : DaTra :=
  (D.DomInc.obj emptyDominion).obj

/-- Initiality in `DaTrav` follows from the left adjoint. Initiality after
forgetting to the wider category `DaTra` is additional data: a wide
subcategory can have fewer arrows, so the latter does not follow formally. -/
structure EmptyMapData {C : CartographyData} (D : DomanialInclusionData C) where
  isInitial : IsInitial (emptyMap D)

def emptyMap_isInitial {C : CartographyData} {D : DomanialInclusionData C}
    (h : EmptyMapData D) : IsInitial (emptyMap D) := h.isInitial

/-- The bifunctor and its strict-unit comparison. The universal-arrow
construction described in the prose is exactly the missing constructor of
this record. -/
structure HorizontalData {C : CartographyData} (D : DomanialInclusionData C) where
  HorSum : Functor (Prod DaTrav DaTrav) DaTrav
  leftUnit : forall X, Nonempty
    (Iso (HorSum.obj (D.DomInc.obj emptyDominion, X)) X)
  rightUnit : forall X, Nonempty
    (Iso (HorSum.obj (X, D.DomInc.obj emptyDominion)) X)

def HorSum {C : CartographyData} {D : DomanialInclusionData C}
    (H : HorizontalData D) : Functor (Prod DaTrav DaTrav) DaTrav := H.HorSum

theorem horizontalLemma {C : CartographyData} {D : DomanialInclusionData C}
    (H : HorizontalData D) : Nonempty (HorizontalData D) := Nonempty.intro H

/-- Mathlib's `MonoidalCategory` and `SymmetricCategory` structures contain
the pentagon, triangle, hexagon, naturality, and involutivity proofs. The two
equalities identify their abstract tensor and unit with Datra's terminology. -/
structure DaTravMonData {C : CartographyData} {D : DomanialInclusionData C}
    (H : HorizontalData D) where
  monoidal : MonoidalCategory DaTrav
  symmetric : @SymmetricCategory DaTrav _ monoidal
```

```
  tensor_eq : @MonoidalCategory.tensor DaTrav _ monoidal = H.HorSum
  unitIso : Iso (@MonoidalCategoryStruct.tensorUnit DaTrav _
      monoidal.toMonoidalCategoryStruct) (D.DomInc.obj emptyDominion)

abbrev DaTravMon {C : CartographyData} {D : DomanialInclusionData C}
    (H : HorizontalData D) := DaTravMonData H

def Brd {C : CartographyData} {D : DomanialInclusionData C}
    {H : HorizontalData D} (M : DaTravMon H) (X Y : DaTrav) := by
  letI := M.monoidal
  letI := M.symmetric
  exact BraidedCategory.braiding X Y

def Asoc {C : CartographyData} {D : DomanialInclusionData C}
    {H : HorizontalData D} (M : DaTravMon H) (X Y Z : DaTrav) := by
  letI := M.monoidal
  exact MonoidalCategoryStruct.associator X Y Z

theorem serenityLemma {C : CartographyData} {D : DomanialInclusionData C}
    {H : HorizontalData D} (M : DaTravMon H) :
    Nonempty (@SymmetricCategory DaTrav _ M.monoidal) := Nonempty.intro M.symmetric

end Datra
```

Listing 1: Complete Lean formalization of DaTra